

Week 4 – TypeScript + Git: Detailed Exercises (Days 22-28)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 7 Sep 2026 – 13 Sep 2026 (Days 22–28)

 **Your slots (IST):** Mon–Fri **7:30–8:30pm** & **9:00–10:00pm** · Sat–Sun **9:00am–12:00pm**, **3:00–5:30pm**, **6:00–8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import `calendar/mern-mastery-study.ics`

Roadmap: [ROADMAP.md](#) · Theory: [02-typescript-complete.md](#), [00-git-complete.md](#) · Index: [week-04-exercises.md](#)

Day 22 – TypeScript Setup & Basic Types

Exercise 22.1: Type the processUsers Function

Problem statement

Convert to TypeScript with strict types:

```
function processUsers(users) {  
  return users  
    .filter(u => u.age >= 18)  
    .map(u => ({ id: u.id, name: u.name.toUpperCase() }));  
}
```

Learning objective

Define interfaces, annotate parameters/returns, enable `strict` mode.

Prerequisites

- Theory: [02-typescript-complete.md §1–2](#)

Step-by-step approach

1. `npm init -y && npm i -D typescript @types/node` .
2. `npx tsc --init` with `"strict": true` .
3. Define `interface User { id: string; name: string; age: number; }` .
4. Define return type `{ id: string; name: string; }[]` or alias `ProcessedUser` .
5. Annotate `processUsers(users: User[]): ProcessedUser[]` .
6. Run `tsc --noEmit` until zero errors.
7. Add test call in `index.ts` .

Hints

1. Explicit return type catches map shape mistakes.
2. `readonly` on id optional for immutability discussion.
3. Use `satisfies` (TS 4.9+) for inference + validation optional.

Exact Answer

- Input users with age < 18 filtered out
- Output: `{ id: string; name: string }[]` with names uppercased
- Compiles with `strict: true` , zero implicit `any`

Complete Solution

```
interface User {
  id: string;
  name: string;
  age: number;
}

interface ProcessedUser {
  id: string;
  name: string;
}

function processUsers(users: User[]): ProcessedUser[] {
  return users
    .filter((u) => u.age >= 18)
    .map((u) => ({ id: u.id, name: u.name.toUpperCase() }));
}

const users: User[] = [
  { id: "1", name: "alice", age: 17 },
  { id: "2", name: "bob", age: 25 },
];

console.log(processUsers(users)); // [{ id: "2", name: "BOB" }]
```

Expected output / acceptance criteria

- Compiles with strict; no implicit `any` .
- Filter removes users with age < 18.

Where to practice

- TS Handbook: <https://www.typescriptlang.org/docs/handbook/intro.html>
- Local: `mern-mastery/week-04/ts-basics/`

Common mistakes

- Leaving `any` on users parameter.
- Forgetting to uppercase in map breaks return contract tests.

80 LPA Interview Twist

Google asks you to add branded types (`UserId` vs `OrderId`) and runtime validation with Zod that infers the same TS types — compile-time safety alone is insufficient at scale.

Interview connection

"How do you type API responses in React?" starts with interfaces like User.

Time estimate: 45 minutes

Day 23 – Interfaces vs Types

Exercise 23.1: Define Five Domain Interfaces

Problem statement

For an e-commerce domain, define: `Product` , `CartItem` , `Order` , `Address` , `UserProfile` . Use interface extension where appropriate.

Learning objective

Choose interface vs type alias; compose with `extends` .

Prerequisites

- Theory: [02-typescript-complete.md §3](#)

Step-by-step approach

1. `Product` : id, name, price, category, stock.
2. `CartItem` : productId, quantity, unitPrice snapshot.
3. `Address` : lines, city, postal, country.

4. `UserProfile` `extends` base user with addresses array.
5. `Order` : `userId`, `items`, `status` union, `total`.
6. Export all from `types/commerce.ts` .
7. Write one `type` alias for `OrderStatus = 'pending' | 'paid' | 'shipped'` .

Hints

1. Interfaces merge in declaration merging — types don't.
2. Union statuses as literal types for exhaustiveness.
3. Use `readonly` on id fields.

Exact Answer

- Five interfaces compile and compose via `extends`
- `OrderStatus` as union type enables exhaustiveness checking in switch

Complete Solution

```
type OrderStatus = "pending" | "paid" | "shipped";

interface Product {
  readonly id: string;
  name: string;
  price: number;
  category: string;
  stock: number;
}

interface CartItem {
  productId: string;
  quantity: number;
  unitPrice: number;
}

interface Address {
  line1: string;
  line2?: string;
  city: string;
  postal: string;
  country: string;
}

interface User {
  readonly id: string;
  email: string;
  name: string;
}

interface UserProfile extends User {
  addresses: Address[];
  phone?: string;
}

interface Order {
  readonly id: string;
  userId: string;
  items: CartItem[];
  status: OrderStatus;
  total: number;
  shippingAddress: Address;
}
```

Expected output / acceptance criteria

- Five interfaces compile; used in sample objects without errors.

Where to practice

- Local: `week-04/day-23/interfaces/`

Common mistakes

- Using `interface` for union-only shapes better as `type` .
- Optional everything (`?`) — be intentional.

80 LPA Interview Twist

Meta system design ties domain modeling to API contracts — expect "model an e-commerce checkout" with TypeScript interfaces aligned to REST + GraphQL schema in one whiteboard session.

Interview connection

Modeling domain entities in full-stack apps and Prisma schema alignment.

Time estimate: 50 minutes

Day 24 – Generics

Exercise 24.1: Generic API Wrapper

Problem statement

Implement:

```
interface ApiResponse<T> {  
  data: T;  
  status: number;  
  error?: string;  
}  
  
async function apiCall<T>(url: string): Promise<ApiResponse<T>>
```

Learning objective

Generic functions, fetch typing, error normalization.

Prerequisites

- Theory: [02-typescript-complete.md §5 Generics](#)

Step-by-step approach

1. Use `fetch(url)` ; check `response.ok` .
2. Parse JSON as `T` with generic call site: `apiCall<User[]>('/users')` .

3. Return structured `ApiResponse` ; on failure include status + error message.
4. Add optional `init?: RequestInit` parameter.
5. Test with <https://jsonplaceholder.typicode.com/users/1>.
6. Never use `as T` without validation — mention runtime gap.

Hints

1. Generic `<T>` on function lets caller specify data shape.
2. `response.json()` returns `Promise<any>` — assign to T consciously.
3. Wrap try/catch for network errors.

Exact Answer

- `apiCall<User[]>('/users')` → typed `data: User[]` at call site
- Non-OK HTTP returns `{ data: null, status, error: string }`
- Network errors caught and normalized

Complete Solution

```
interface ApiResponse<T> {
  data: T | null;
  status: number;
  error?: string;
}

async function apiCall<T>(url: string, init?: RequestInit): Promise<ApiResponse<T>
  try {
    const response = await fetch(url, init);
    if (!response.ok) {
      return {
        data: null,
        status: response.status,
        error: `HTTP ${response.status}: ${response.statusText}`,
      };
    }
    const data = (await response.json()) as T;
    return { data, status: response.status };
  } catch (err) {
    return {
      data: null,
      status: 0,
      error: err instanceof Error ? err.message : "Network error",
    };
  }
}

interface User { id: number; name: string; email: string; }

apiCall<User>("https://jsonplaceholder.typicode.com/users/1")
  .then((res) => console.log(res.data?.name)); // typed as string | undefined
```

Line-by-line explanation (typescript)

Line	What it does
<code>apiCall<User>("https:</code>	<code>jsonplaceholder.typicode.com/users/1")</code>
<code>.then((res) =></code> <code>console.log(res.data?.name));</code>	typed as string

Expected output / acceptance criteria

- Typed `data` at call site without manual cast.
- Non-OK HTTP returns error field.

Where to practice

- JSONPlaceholder: <https://jsonplaceholder.typicode.com/>

- Local: `week-04/day-24/apiCall.ts`

Common mistakes

- Generics on interface property incorrectly (`ApiResponse<T extends ...>` confusion).
- Swallowing fetch errors.

80 LPA Interview Twist

Senior rounds ask for generic retry, timeout, and Zod runtime parse of `T` — "trust boundary at the network edge" is a Google L4 talking point.

Interview connection

Axios wrappers and React Query `useQuery<T>` patterns.

Time estimate: 55 minutes

Day 25 – Utility Types & Type Guards

Exercise 25.1: Product Utility Types

Problem statement

Given `Product` interface, create:

- `ProductCreate` — omit id, createdAt
- `ProductUpdate` — partial of ProductCreate
- `ProductPublic` — omit internal fields (cost, supplierId)

Learning objective

Use `Omit`, `Partial`, `Pick` for DTO layers.

Prerequisites

- Theory: [02-typescript-complete.md §6–7](#)

Step-by-step approach

1. Define full `Product` with internal + public fields.
2. `type ProductCreate = Omit<Product, 'id' | 'createdAt'> .`
3. `type ProductUpdate = Partial<ProductCreate> .`
4. `type ProductPublic = Omit<Product, 'cost' | 'supplierId'> .`
5. Write functions accepting each type for create/update/read API.
6. Verify incompatible fields fail at compile time.

Hints

1. `Partial` makes all keys optional — ideal for PATCH.
2. `Pick` alternative if public fields are fewer than private.
3. Combine `Partial<Omit<...>>` carefully.

Exact Answer

- `ProductCreate` omits `id`, `createdAt`
- `ProductUpdate` is partial of create DTO
- `ProductPublic` omits internal `cost`, `supplierId`
- Compile error if `cost` passed to public endpoint

Complete Solution

```
interface Product {
  readonly id: string;
  name: string;
  price: number;
  cost: number;
  supplierId: string;
  createdAt: Date;
}

type ProductCreate = Omit<Product, "id" | "createdAt">;
type ProductUpdate = Partial<ProductCreate>;
type ProductPublic = Omit<Product, "cost" | "supplierId">;

function createProduct(data: ProductCreate): Product {
  return { ...data, id: crypto.randomUUID(), createdAt: new Date() };
}

function updateProduct(id: string, data: ProductUpdate): void {
  console.log("PATCH", id, data);
}

function toPublic(product: Product): ProductPublic {
  const { cost, supplierId, ...pub } = product;
  return pub;
}

// createProduct({ name: "X", price: 10, cost: 5, supplierId: "s1" }); // OK
// toPublic(...).cost; // compile error
```

Expected output / acceptance criteria

- TS errors when passing `cost` to public endpoint function.

Where to practice

- TS Utility Types: <https://www.typescriptlang.org/docs/handbook/utility-types.html>
- Local: `week-04/day-25/utilities.ts`

Common mistakes

- Duplicating interfaces manually instead of utility composition.
- Update type allowing id change.

80 LPA Interview Twist

Amazon API design asks for DTO layers matching OpenAPI spec with auto-generated types — utility type composition must scale to 50+ entity models without manual duplication.

Interview connection

API layer typing in Catalog project; matches Zod schema inference later.

Time estimate: 40 minutes

Exercise 25.2: Type Guard `isAdmin`

Problem statement

```
function isAdmin(user: User | Admin): user is Admin {  
  // implement  
}
```

Learning objective

Narrow discriminated unions with type predicates.

Step-by-step approach

1. Add discriminant: `role: 'user' | 'admin'` or `'permissions' in user`.
2. Return boolean predicate `user is Admin`.
3. Use in `if (isAdmin(u)) { u.adminPanel }` — must typecheck.
4. Alternative: switch on `user.role`.

Hints

1. `user is Admin` return type enables narrowing.
2. Prefer discriminated union over optional fields.
3. `in` operator narrows in TS.

Exact Answer

- `isAdmin(user)` returns `true` only when `role === 'admin'`
- Inside `if (isAdmin(u))` block, `u.adminPanel` typechecks without error

Complete Solution

```
interface User {
  id: string;
  name: string;
  role: "user";
}

interface Admin {
  id: string;
  name: string;
  role: "admin";
  adminPanel: string;
}

function isAdmin(user: User | Admin): user is Admin {
  return user.role === "admin";
}

function handleUser(u: User | Admin) {
  if (isAdmin(u)) {
    console.log(u.adminPanel); // TS knows u is Admin
  } else {
    console.log(u.name);
  }
}
```

Expected output / acceptance criteria

- Branch accesses admin-only fields without error.

80 LPA Interview Twist

Google asks for exhaustiveness checking with `never` in switch on discriminated unions, plus migrating from type guards to Zod `.safeParse` at API boundaries.

Where to practice

- Local: `week-04/day-25/type-guards.ts`

Time estimate: 30 minutes

Exercise 25.3: Discriminated Union Result

Problem statement

```
type Result<T> =  
  | { success: true; data: T }  
  | { success: false; error: string };  
  
function handleResult<T>(result: Result<T>): T
```

Learning objective

Exhaustive narrowing; throw or return on failure branch.

Step-by-step approach

1. Check `result.success` ; if true return `result.data` .
2. Else throw `new Error(result.error)` or return never.
3. Test both branches compile.
4. Use in service layer wrapping DB calls.

Hints

1. Switch on `success` for exhaustiveness.
2. Never access `data` when success false — TS should error.
3. Same pattern as Rust Result / API envelopes.

Exact Answer

- `handleResult({ success: true, data: T })` → returns `T`
- `handleResult({ success: false, error })` → throws `Error`
- TypeScript narrows correctly on `success` discriminant

Complete Solution

```
type Result<T> =  
  | { success: true; data: T }  
  | { success: false; error: string };  
  
function handleResult<T>(result: Result<T>): T {  
  if (result.success) {  
    return result.data;  
  }  
  throw new Error(result.error);  
}  
  
const ok: Result<number> = { success: true, data: 42 };  
const fail: Result<number> = { success: false, error: "Not found" };  
  
console.log(handleResult(ok)); // 42  
// console.log(handleResult(fail)); // throws Error("Not found")
```

Expected output / acceptance criteria

- `handleResult` returns `T`; throws on failure case in tests.

80 LPA Interview Twist

Meta service layer uses Result types instead of exceptions — expect "refactor this try/catch API to `Result<T,E>`" with full generic error typing and railway-oriented composition.

Where to practice

- Local: `week-04/day-25/result.ts`

Time estimate: 25 minutes

Day 26 – LeetCode Binary Search

Exercise 26.1: LC 704 – Binary Search

Problem statement

Search target in sorted array; return index or -1.

Learning objective

Classic binary search invariant: `left <= right`.

Prerequisites

- Theory: [08-dsa-patterns-complete.md](#)

Step-by-step approach

1. `left = 0` , `right = nums.length - 1` .
2. While `left <= right` , `mid = left + Math.floor((right-left)/2)` .
3. Compare `nums[mid]` with target; adjust bounds.
4. Avoid overflow with `left + (right-left)/2` .
5. Return -1 if not found.

Exact Answer

- `search([-1,0,3,5,9,12], 9) → 4`
- Time: $O(\log n)$, Space: $O(1)$

Complete Solution

```
function search(nums: number[], target: number): number {
  let left = 0;
  let right = nums.length - 1;

  while (left <= right) {
    const mid = left + Math.floor((right - left) / 2);
    if (nums[mid] === target) return mid;
    if (nums[mid] < target) left = mid + 1;
    else right = mid - 1;
  }
  return -1;
}

console.log(search([-1, 0, 3, 5, 9, 12], 9)); // 4
```

Expected output / acceptance criteria

- `search([-1,0,3,5,9,12], 9) → 4`

80 LPA Interview Twist

Google extends to LC 33 rotated array and LC 875 binary search on answer space — expect lower-bound template (`while left < right`) derived from first principles in 15 minutes.

Where to practice

- <https://leetcode.com/problems/binary-search/>

Time estimate: 30 minutes

Day 27 – Git Workflow Exercises

Exercise 27.1: Branch, Merge, Conflict Resolution

Problem statement

Complete Git workflow:

1. Init repo `mern-mastery` (if not exists)
2. Branches: `main` , `week-04-ts-catalog`
3. Five meaningful commits
4. Intentional merge conflict → resolve
5. `.gitignore` for Node.js

Learning objective

Fluency with branch workflow, conflict markers, atomic commits.

Prerequisites

- Theory: [00-git-complete.md](#)

Step-by-step approach

1. `git init` or clone; create `.gitignore` (`node_modules`, `dist`, `.env`).
2. `git checkout -b week-04-ts-catalog` .
3. Commit catalog API in logical chunks: setup, types, routes, validation, README.
4. On `main` , edit same line in README as on branch; merge branch → conflict.
5. Resolve `<<<<<<<` markers; `git add` ; `git commit` .
6. Push to GitHub: `git remote add origin ... && git push -u origin main` .

Hints

1. Commit messages: imperative mood ("Add product schema").
2. `git status` before every commit.
3. Never commit `.env` or secrets.

Exact Answer

- 5+ commits on `week-04-ts-catalog` branch with imperative messages
- Merge conflict resolved cleanly (no leftover `<<<<<<<` markers)
- `.gitignore` excludes `node_modules` , `dist` , `.env`

Complete Solution

```
git init
echo "node_modules/\ndist/\n.env" > .gitignore
git checkout -b week-04-ts-catalog
git add .
git commit -m "Add TypeScript catalog API setup"
git commit -m "Add Product types and Zod schema"
git commit -m "Add CRUD routes"
git commit -m "Add validation middleware"
git commit -m "Add README and API docs"
git checkout main
# Edit README on main to create conflict, then:
git merge week-04-ts-catalog
# Resolve conflict markers, then:
git add . && git commit -m "Merge week-04-ts-catalog with conflict resolution"
git push -u origin main
```

Expected output / acceptance criteria

- Clean `git log --oneline` with 5+ commits on feature branch.
- Successful merge after conflict resolution.
- GitHub repo visible with both branches.

Where to practice

- GitHub new repo: <https://github.com/new>
- Git docs: <https://git-scm.com/doc>
- Learn Git Branching: <https://learngitbranching.js.org/>

Common mistakes

- Committing node_modules.
- Deleting conflict markers without choosing correct content.
- Force pushing main.

80 LPA Interview Twist

Meta/Google ask "walk me through rebasing vs merging a feature branch" and "how would you revert a bad commit in production" — Git fluency beyond init/commit/push is expected at 80 LPA.

Interview connection

"Describe your Git workflow" in behavioral + practical pair programming.

Time estimate: 3 hours

Days 27-28 – TypeScript Catalog API Project

Project: In-Memory Catalog REST API

Problem statement

Build Express + TypeScript API:

- `GET/POST/PUT/DELETE /products`
- Zod validation on create/update
- Typed request/response; in-memory store

Learning objective

End-to-end typed backend before Month 3 Node deep dive.

Prerequisites

- Theory: [02-typescript-complete.md](#)
- [05-nodejs-express-complete.md §4–5](#) preview

Step-by-step approach

1. **Scaffold:** `npm i express zod`, `npm i -D typescript tsx @types/express`. `src/server.ts`, `src/types/product.ts`.
2. **Model:** `Product` interface + Zod schema `ProductCreateSchema` inferred type.
3. **Store:** `let products: Product[] = []` in service module.
4. **Routes:** Express router `/products` — list, get by id, create, update, delete.
5. **Validation middleware:** parse body with Zod; return 400 with issues array.
6. **Error handler:** typed middleware `(err, req, res, next)`.
7. **Scripts:** `"dev": "tsx watch src/server.ts"`.
8. **Test manually:** curl or Thunder Client — document in README.

Hints

1. `z.infer<typeof Schema>` keeps runtime + compile sync.
2. Use `crypto.randomUUID()` for ids on POST.
3. Return 404 for missing id consistently.

Exact Answer

- `GET /products` → JSON array of products
- `POST /products` with invalid body → 400 with Zod error details
- `PUT/DELETE /products/:id` → 404 if missing

- Server runs on `localhost:3000` with `strict: true` compile

Complete Solution

```
// src/types/product.ts
import { z } from "zod";

export const ProductCreateSchema = z.object({
  name: z.string().min(1),
  price: z.number().positive(),
  category: z.string(),
  stock: z.number().int().nonnegative(),
});

export type ProductCreate = z.infer<typeof ProductCreateSchema>;

export interface Product extends ProductCreate {
  id: string;
  createdAt: string;
}

// src/store.ts
import { Product } from "../types/product.js";
export const products: Product[] = [];

// src/routes/products.ts
import { Router } from "express";
import { randomUUID } from "crypto";
import { ProductCreateSchema } from "../types/product.js";
import { products } from "../store.js";

const router = Router();

router.get("/", (_req, res) => res.json(products));

router.get("/:id", (req, res) => {
  const p = products.find((x) => x.id === req.params.id);
  if (!p) return res.status(404).json({ error: "Not found" });
  res.json(p);
});

router.post("/", (req, res) => {
  const parsed = ProductCreateSchema.safeParse(req.body);
  if (!parsed.success) return res.status(400).json({ errors: parsed.error.issues });
  const product = { ...parsed.data, id: randomUUID(), createdAt: new Date().toISOString() };
  products.push(product);
  res.status(201).json(product);
});

router.put("/:id", (req, res) => {
  const idx = products.findIndex((x) => x.id === req.params.id);
  if (idx === -1) return res.status(404).json({ error: "Not found" });
  const parsed = ProductCreateSchema.partial().safeParse(req.body);
  if (!parsed.success) return res.status(400).json({ errors: parsed.error.issues });
  products[idx] = { ...products[idx], ...parsed.data };
});
```

```
    res.json(products[idx]);
  });

  router.delete("/:id", (req, res) => {
    const idx = products.findIndex((x) => x.id === req.params.id);
    if (idx === -1) return res.status(404).json({ error: "Not found" });
    products.splice(idx, 1);
    res.status(204).send();
  });

  export default router;

// src/server.ts
import express from "express";
import productsRouter from "../routes/products.js";

const app = express();
app.use(express.json());
app.use("/products", productsRouter);

app.use((err: Error, _req: express.Request, res: express.Response, _next: express.NextFunction) => {
  console.error(err);
  res.status(500).json({ error: "Internal server error" });
});

app.listen(3000, () => console.log("Catalog API on :3000"));
```

Expected output / acceptance criteria

- `GET /products` → JSON array
- `POST /products` invalid body → 400 with Zod errors
- `strict: true` compile clean
- Runs on `localhost:3000`

Where to practice

- Zod: <https://zod.dev/>
- Express TS: <https://expressjs.com/>
- Local: `mern-mastery/week-04/catalog-api/`

Common mistakes

- Parsing body as `any` .
- No centralized error handler (stack traces leak in prod).
- Forgetting `express.json()` middleware.

80 LPA Interview Twist

Amazon L6 expects Catalog API extended with pagination, auth middleware, OpenAPI spec

generation, and integration tests — the in-memory store is a scaffold for discussing production architecture tradeoffs.

Interview connection

Mini version of production REST services; validation + typing is senior expectation.

Time estimate: 6–8 hours

Exercise 28.1: LC 35 & LC 278 (Binary Search Variants)

Problem statement

Solve Search Insert Position (LC 35) and First Bad Version (LC 278).

Learning objective

Binary search on answer space and lower-bound template.

Step-by-step approach

1. LC 35: standard BS; if not found return `left` insertion point.
2. LC 278: `isBadVersion` API — minimize first bad index.
3. Document lower-bound template reusable for both.

Exact Answer

- LC 35: `searchInsert([1,3,5,6], 5) → 2 ; searchInsert([1,3,5,6], 2) → 1`
- LC 278: minimize first bad version using binary search on `[1, n]`
- Both: Time $O(\log n)$, Space $O(1)$

Complete Solution

```
// LC 35 - Search Insert Position
function searchInsert(nums: number[], target: number): number {
  let left = 0, right = nums.length;
  while (left < right) {
    const mid = left + Math.floor((right - left) / 2);
    if (nums[mid] < target) left = mid + 1;
    else right = mid;
  }
  return left;
}

// LC 278 - First Bad Version
function firstBadVersion(isBadVersion: (v: number) => boolean): (n: number) => number {
  return function (n: number): number {
    let left = 1, right = n;
    while (left < right) {
      const mid = left + Math.floor((right - left) / 2);
      if (isBadVersion(mid)) right = mid;
      else left = mid + 1;
    }
    return left;
  };
}

console.log(searchInsert([1, 3, 5, 6], 5)); // 2
console.log(searchInsert([1, 3, 5, 6], 2)); // 1
```

Line-by-line explanation (typescript)

Line	What it does
<code>console.log(searchInsert([1, 3, 5, 6], 5));</code>	2
<code>console.log(searchInsert([1, 3, 5, 6], 2));</code>	1

Where to practice

- <https://leetcode.com/problems/search-insert-position/>
- <https://leetcode.com/problems/first-bad-version/>

Time estimate: 45 minutes each

80 LPA Interview Twist

Lower-bound binary search template solves 20+ LeetCode mediums — Google interviewers ask you to derive it from scratch and apply to "find first position" variants without memorizing LC numbers.

Exercise 28.2: Month 1 Review – 50 JS Flashcards

Problem statement

Create 50 flashcards (Anki or markdown) covering Weeks 1–4: hoisting, closures, this, prototypes, async, TS utilities, Git commands.

Exact Answer

- 50 Q/A pairs in `month-01-flashcards.md`
- Self-test score $\geq 80\%$ (40/50 correct)
- Cover all 7 core JS topics + TS utilities + Git commands

Complete Solution

Sample flashcards (create 50 total)

Q: What is the output order of sync vs microtask vs macrotask?

A: Sync first, then microtasks (Promises), then macrotasks (setTimeout).

Q: `typeof null`?

A: "object" (historical bug).

Q: What does `Omit<T, K>` do?

A: Creates type with keys K removed from T.

Q: git merge vs git rebase?

A: Merge preserves branch history; rebase replays commits linearly.

Q: Two Sum optimal complexity?

A: $O(n)$ time, $O(n)$ space with hash map.

Expected output / acceptance criteria

- File `month-01-flashcards.md` with Q/A pairs.
- Self-test score $\geq 80\%$.

80 LPA Interview Twist

Flashcards become rapid-fire phone screen prep — Google recruiters expect you to answer any Month 1 topic in under 30 seconds with an example, not just definition recall.

Time estimate: 2 hours

Week 4 Completion Checklist

Item	Done
All TS exercises strict compile	☐
Git conflict resolved	☐
Catalog API CRUD + Zod	☐
LC 704, 35, 278	☐
Month 1 flashcards	☐

Total estimated time: ~24–28 hours